


ΣΧΟΛΗ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ ΚΑΙ ΜΗΧΑΝΙΚΩΝ ΥΠΟΛΟΓΙΣΤΩΝ
Ε.Μ.Π
Π.Μ.Σ: ΕΠΙΣΤΗΜΗ ΔΕΔΟΜΕΝΩΝ ΚΑΙ ΜΗΧΑΝΙΚΗ ΜΑΘΗΣΗ

# ΕΞΑΜΗΝΙΑΙΑ ΕΡΓΑΣΙΑ

## ΔΙΑΧΕΙΡΙΣΗ ΔΕΔΟΜΕΝΩΝ ΜΕΓΑΛΗΣ ΚΛΙΜΑΚΑΣ/ BIG DATA

ΟΝΟΜΑΤΕΠΩΝΥΜΟ: **ΚΑΡΑΒΑΓΓΕΛΗΣ ΑΝΤΩΝΗΣ**

ΙΟΥΝΙΟΣ 2026

# Πίνακας Περιεχομένων

Ζητούμενο 1ο: Αξιολόγηση Formats Αποθήκευσης . . . . . 2
Ζητούμενο 2ο: Υλοποίηση Query 1 . . . . . 3
Ζητούμενο 3ο: Υλοποίηση Query 2 . . . . . 4
Ζητούμενο 4ο: Υλοποίηση Query 3 . . . . . 6
Ζητούμενο 5ο: Υλοποίηση Query 4 . . . . . 7
Ζητούμενο 6ο: Ανάλυση του Catalyst Optimizer και Στρατηγικές Join για τα Queries 3,4 . . . . . 10

## Ζητούμενο 1ο: Αξιολόγηση Formats Αποθήκευσης

Τα αρχικά δεδομένα της παρούσας εργασίας είναι αποθηκευμένα σε μορφή κειμένου CSV (Comma-Separated Values). Αν και το CSV αποτελεί ένα εξαιρετικά διαδεδομένο format λόγω της αναγνωσιμότητάς του από τον άνθρωπο,δεν αποτελεί το καταλληλότερο format για την επεξεργασία μεγάλων δεδομένων από μηχανές όπως το Apache Spark. Τα βασικά μειονεκτήματα του CSV είναι τα εξής:

- Row-oriented): Το CSV αποθηκεύει τα δεδομένα ανά γραμμή. Αν ένα ερώτημα απαιτεί την προσπέλαση 2 στηλών από τις 30, το Spark αναγκάζεται να εκτελέσει full scan σε ολόκληρο το αρχείο από τον δίσκο, σπαταλώντας άσκοπα πόρους.
- Έλλειψη Σχήματος (Schema): Δεν αποθηκεύονται μεταδεδομένα για τους τύπους των στηλών, αναγκάζοντας το Spark είτε να εκτελεί διπλό πέρασμα του αρχείου είτε να μετατρέπει τα δεδομένα χειροκίνητα με cast.
- Δεν υποστηρίζει ενσωματωμένη συμπίεση ή indexing, αυξάνοντας τον αποθηκευτικό χώρο στο HDFS.

Τα βασικά εναλλακτικά formats στον κόσμο των Big Data είναι κυρίως το Apache Parquet και το το Apache ORC το οποίο είναι column oriented αλλά ενδείκνυται περισσότερο για το οικοσύστημα του Apache Hive. Το format που επιλέχθηκε για την σύγκριση ήταν το Apache Parquet, καθώς αποτελεί το πλέον optimized format για το Spark.

### Πειραματική Σύγκριση Επίδοσης (CSV vs Parquet στο Query 1)

Τα αρχικά CSV αρχεία των εγκλημάτων (`2010_2019.csv` και `2020_2025.csv`) μετατράπηκαν σε Parquet και αποθηκεύτηκαν στον προσωπικό κατάλογο του HDFS (`/user/dsml00292/LA_Crime_Data_Parquet/`). Στη συνέχεια, εκτελέστηκε το Query 1 και με τα δύο φορμάτ υπό τις ίδιες συνθήκες cluster (3 executors, 1 core, 2 GB μνήμης).

Στον Πίνακα 1 αποτυπώνονται οι συγκριτικοί χρόνοι εκτέλεσης για τις τρεις προσεγγίσεις του ερωτήματος.

Table 1: Συγκριτικοί χρόνοι εκτέλεσης (CSV vs Apache Parquet) στο Query 1

| **Υλοποίηση Query 1** | **Χρόνος CSV (s)** | **Χρόνος Parquet (s)** | **Βελτίωση (Speedup)** |
|---|---|---|---|
| DataFrame API (without UDF) | 41.89 | 9.72 | 4.31× |
| DataFrame API (with UDF) | 56.87 | 7.03 | 8.09× |
| RDD API | 89.91 | 28.99 | 3.10× |

Η εντυπωσιακή μείωση των χρόνων εκτέλεσης με τη χρήση του Parquet ερμηνεύεται βάσει των εξής μηχανισμών:

- Column Pruning (Αποκοπή Στηλών): Το Query 1 χρησιμοποιεί μόνο τις στήλες `TIME OCC` και `Premis Desc`. Κατά την ανάγνωση του αρχείου Parquet, το Spark προσπέλασε αποκλειστικά τα bytes αυτών των δύο στηλών στο δίσκο, αγνοώντας πλήρως τις υπόλοιπες στήλες του dataset. Αυτό μείωσε δραματικά το Network και Disk I/O.
- Εξάλειψη του Schema Inference Overhead: Το Parquet αποθηκεύει το Schema στα metadata του αρχείου. Έτσι, το Spark γνώριζε εκ των προτέρων τους τύπους δεδομένων, αποφεύγοντας το αρχικό σκανάρισμα που απαιτούσε το CSV.

## Ζητούμενο 2ο: Υλοποίηση Query 1

Κατά την εκτέλεση του Query 1 με τη χρήση 2 executors (1 core και 2 GB μνήμης ανά executor), επαληθεύτηκε όπως ήταν λογικό πως και οι τρεις διαφορετικές προσεγγίσεις παρήγαγαν ακριβώς τα ίδια αποτελέσματα. Συγκεκριμένα, ο διαχωρισμός των εγκλημάτων, ο συνολικός αριθμός τους, καθώς και το ποσοστό των εγκλημάτων που διαπράχθηκαν στον δρόμο (*street*) ανά χρονική περίοδο (NIGHT, EVENING,MORNING,AFTERNOON) συμπίπτουν απόλυτα και στις τρεις μεθόδους.

Στον Πίνακα 2 παρουσιάζονται τα κοινά αποτελέσματα της κατανομής των εγκλημάτων, όπως αυτά εξήχθησαν από το cluster:

Table 2: Κατανομή και ποσοστά εγκλημάτων ανά χρονική περίοδο (Query 1 Results)

| Part of the Day | Total Crimes | Street Crimes | Street Percentage |
|---|---|---|---|
| NIGHT | 870,235 | 251,094 | 28.8536% |
| EVENING | 719,695 | 198,292 | 27.5522% |
| MORNING | 693,099 | 130,866 | 18.8813% |
| AFTERNOON | 855,099 | 156,432 | 18.2940% |

Η διαφορά των τριών μεθόδων εντοπίζεται στη σημαντική απόκλιση στους χρόνους εκτέλεσης της κάθε προσέγγισης. Αυτοί οι χρόνοι, παρουσιάζονται συγκεντρωτικά παρακάτω.

Table 3: Χρόνοι εκτέλεσης και σχετική καθυστέρηση υλοποιήσεων Query 1

| Μέθοδος | Χρόνος Εκτέλεσης (s) | Καθυστέρηση |
|---|---|---|
| DataFrame API (without UDF) | 41.89 | — |
| DataFrame API (with UDF) | 56.87 | +35.7% |
| RDD API | 89.91 | +114.6% |

Όπως παρατηρείται από τον παραπάνω πίνακα η χρήση DataFrame API χωρίς User Difined Functions (41.89 s): αποτελεί την ταχύτερη υλοποίηση. Όταν χρησιμοποιούνται native συναρτήσεις του Spark (όπως οι `hour()` και `when()`), ο *Catalyst Optimizer* έχει την πλήρη εποπτεία του logical plan. Έτσι, πραγματοποιεί βελτιστοποιήσεις, ενώ η μηχανή *Tungsten* παράγει ταχύτατο bytecode απευθείας στην JVM (*Whole-Stage Code Generation*), διατηρώντας τα δεδομένα σε binary μορφή (off-heap memory) χωρίς το overhead δημιουργίας αντικειμένων Java.Αν τώρα χρησιμοποιήσουμε DataFrame API αλλά με UDF (56.87 s) τότε παρατηρούμε ξεκάθαρη καθυστέρηση της τάξης του +35.7%. Η custom συνάρτηση λειτουργεί ως black box για τον Catalyst Optimizer, εμποδίζοντας οποιαδήποτε δομική βελτιστοποίηση. Επιπλέον, εισάγεται το overhead του context Switching. Για κάθε εγγραφή, τα δεδομένα πρέπει να γίνουν serialize στην JVM, να σταλούν μέσω sockets στον Python worker process για να εκτελεστεί η UDF, και στη συνέχεια να γίνουν ξανά serialize για να επιστρέψουν στην JVM. Όλη αυτή η διαδικασία είναι ιδιαίτερα κοστοβόρα και αιτιολογεί πλήρως την αύξηση του χρόνου εκτέλεσης. Τέλος το RDD API (89.91 s) αποτελεί μακράν την πιο αργή προσέγγιση, απαιτώντας υπερδιπλάσιο χρόνο σε σχέση με το native DataFrame. Τα RDDs στερούνται πλήρως των αυτοματοποιημένων βελτιστοποιήσεων του Catalyst Optimizer και της διαχείρισης μνήμης του Tungsten. Κάθε record αντιμετωρίζεται ως ξεχωριστό logical object, προκαλώντας συνεχή garbage collection, τεράστιο memory overhead και εξαιρετικά αργές διαδικασίες shuffle κατά την εκτέλεση του reduceByKey.

Συμπέρασματικά τα ευρήματα επιβεβαιώνουν τη βέλτιστη πρακτική στο οικοσύστημα του PySpark, σύμφωνα με την οποία πρέπει να αποφεύγεται η χρήση RDDs και custom UDFs όταν η ζητούμενη λογική μπορεί να εκφραστεί με native συναρτήσεις του DataFrame API!

## Ζητούμενο 3ο: Υλοποίηση Query 2

Το Query 2 όπως ζητήθηκε εκτελέστηκε στο cluster με τη χρήση 4 executors (1 core και 2 GB μνήμης ανά executor). Όπως αναμενόταν, τόσο η native DataFrame υλοποίηση όσο και το SQL API παρήγαγαν ακριβώς τα ίδια αποτελέσματα. Και οι δύο προσεγγίσεις ταξινόμησαν με απόλυτη ακρίβεια τους τρεις μήνες με την υψηλότερη εγκληματικότητα για κάθε έτος από το 2010 έως το 2025. Παρακάτω παρουσιάζονται τα πλήρη και κοινά αποτελέσματα της κατάταξης των μηνών ανά έτος.

Table 4: Top 3 Μήνες με την υψηλότερη εγκληματικότητα ανά έτος (2010-2025)

| year | month | total_crimes | ranking |
|---|---|---|---|
| 2010 | 1 | 19,524 | 1 |
| 2010 | 3 | 18,131 | 2 |
| 2010 | 7 | 17,857 | 3 |
| 2011 | 1 | 18,144 | 1 |
| 2011 | 7 | 17,284 | 2 |
| 2011 | 10 | 17,035 | 3 |
| 2012 | 1 | 17,958 | 1 |
| 2012 | 8 | 17,662 | 2 |
| 2012 | 5 | 17,504 | 3 |
| 2013 | 8 | 17,442 | 1 |
| 2013 | 1 | 16,833 | 2 |
| 2013 | 7 | 16,645 | 3 |
| 2014 | 10 | 17,337 | 1 |
| 2014 | 7 | 17,260 | 2 |
| 2014 | 12 | 17,201 | 3 |
| 2015 | 8 | 15,044 | 1 |
| 2015 | 10 | 14,934 | 2 |
| 2015 | 7 | 14,852 | 3 |
| 2016 | 12 | 26,236 | 1 |
| 2016 | 10 | 25,008 | 2 |
| 2016 | 8 | 24,635 | 3 |
| 2017 | 10 | 20,442 | 1 |
| 2017 | 7 | 20,202 | 2 |
| 2017 | 1 | 19,866 | 3 |
| 2018 | 5 | 19,979 | 1 |
| 2018 | 7 | 19,882 | 2 |
| 2018 | 8 | 19,769 | 3 |
| 2019 | 7 | 19,132 | 1 |
| 2019 | 8 | 18,994 | 2 |
| 2019 | 3 | 18,877 | 3 |
| 2020 | 1 | 18,576 | 1 |
| 2020 | 2 | 17,284 | 2 |
| 2020 | 5 | 17,230 | 3 |
| 2021 | 10 | 19,343 | 1 |
| 2021 | 7 | 18,690 | 2 |
| 2021 | 8 | 18,398 | 3 |
| 2022 | 5 | 20,467 | 1 |
| 2022 | 10 | 20,335 | 2 |
| 2022 | 6 | 20,273 | 3 |
| 2023 | 10 | 20,122 | 1 |
| 2023 | 8 | 20,082 | 2 |
| 2023 | 1 | 19,970 | 3 |
| 2024 | 1 | 18,926 | 1 |
| 2024 | 2 | 17,394 | 2 |
| 2024 | 3 | 16,293 | 3 |
| 2025 | 2 | 44 | 1 |
| 2025 | 1 | 26 | 2 |
| 2025 | 3 | 24 | 3 |

Οι χρόνοι εκτέλεσης για τις δύο προσεγγίσεις καταγράφονται στον παρακάτω πίνακα.

Table 5: Σύγκριση χρόνων εκτέλεσης υλοποιήσεων Query 2

| **Προσέγγιση** | **Χρόνος Εκτέλεσης (s)** |
|---|---|
| DataFrame API | 72.12 |
| SQL API | 76.92 |

Η ανάλυση των μετρήσεων δείχνει ότι οι χρόνοι εκτέλεσης μεταξύ του DataFrame API (72.12 s) και του SQL API (76.92 s) είναι ουσιαστικά ισοδύναμοι, με τη μικρή διαφορά των 4.8 s να οφείλεται αποκλειστικά στα όρια του στατιστικού σφάλματος, του δικτυακού overhead και των διακυμάνσεων φόρτου της συστοιχίας (cluster). Η συμπεριφορά αυτή είναι απόλυτα αναμενόμενη βάσει της εσωτερικής αρχιτεκτονικής του Apache Spark. Και στις δύο μεθόδους, είτε ο κώδικας εκφραστεί μέσω native μεθόδων των DataFrames (.select(), .groupBy()), είτε ως αυτούσιο string ερωτήματος SQL μέσω της spark.sql(), το τελικό πλάνο εκτέλεσης παραμένει κοινό. Ο Catalyst Optimizer αναλαμβάνει να μεταφράσει και να βελτιστοποιήσει τις εντολές στο ίδιο ακριβώς logical και physical plan. Κατά συνέπεια, δεν υπάρχει καμία διαφοροποίηση στο επίπεδο επεξεργασίας των δεδομένων από τους executors. Άρα, η επιλογή μεταξύ DataFrame API και SQL API στο PySpark αποτελεί καθαρά ζήτημα προτίμησης, καθώς οι δύο βιβλιοθήκες προσφέρουν σχεδόν την ίδια απόδοση.

## Ζητούμενο 4ο: Υλοποίηση Query 3

Το Query 3 εκτελέστηκε με επιτυχία στο Kubernetes cluster χρησιμοποιώντας 3 executors με 1 core και 2 GB μνήμης ανά executor. Μετά την αφαίρεση των περιορισμών μορφοποίησης στην Python, οι δύο πίνακες αποτελεσμάτων ταυτίστηκαν απόλυτα χαρακτήρα προς χαρακτήρα, εμφανίζοντας το πλήρες βάθος των δεκαδικών ψηφίων. Στον Πίνακα 6 παρουσιάζονται ενδεικτικά οι πρώτες και οι τελευταίες γραμμές των κοινών αποτελεσμάτων, ταξινομημένες φθίνουσα ως προς το κατά κεφαλήν εισόδημα (Income per Capita).

Table 6: Αποτελέσματα Query 3: Κατά κεφαλήν εισόδημα ανά Zip Code στο Los Angeles

| **Zip Code** | **Total Population** | **Income per Capita ($)** |
|---|---|---|
| 90740 | 11 | 6819.909090909091 |
| 93563 | 143 | 1472.7272727272727 |
| 91759 | 74 | 1374.162162162162 |
| 91210 | 555 | 190.15495495495495 |
| 91008 | 1010 | 165.84158415841586 |
| 91362 | 1125 | 113.13688888888889 |
| 93544 | 1126 | 65.19449378330373 |
| 93532 | 2373 | 33.86304256215761 |
| 90067 | 3344 | 32.80622009569378 |
| 93553 | 1921 | 31.132222800624675 |
| ... | ... | ... |
| 90250 | 97653 | 0.675196870551852 |
| 93550 | 82379 | 0.6639192027094284 |
| 90003 | 72764 | 0.6559974712770051 |
| 92678 | 92678 | 0.6443060920606832 |
| 93535 | 82149 | 0.6276400199637244 |
| 90805 | 95350 | 0.6002307288935501 |
| 90201 | 95810 | 0.5172111470618933 |
| 90011 | 102308 | 0.4606286898385268 |
| 90044 | 94354 | 0.4598427199694767 |

Στον Πίνακα 7 παρουσιάζονται συγκεντρωτικά οι τελικοί χρόνοι εκτέλεσης για την κάθε αρχιτεκτονική προσέγγιση.

Table 7: Τελικοί χρόνοι εκτέλεσης υλοποιήσεων Query 3 (3 Executors, 1 Core, 2 GB RAM)

| Αρχιτεκτονική / API | Χρόνος Εκτέλεσης (seconds) | Κατάσταση |
|---|---|---|
| DataFrame API | 49.58 | Success |
| RDD API | 24.24 | Success |

Η επεξεργασία του Query 3 ανέδειξε ορισμένα πολύ κρίσιμα χαρακτηριστικά της εσωτερικής αρχιτεκτονικής του Apache Spark. Αρχικά το αρχείο GeoJSON μεγέθους 180 MB περιέχει σύνθετες spatial δομές. Η απευθείας line-by-line επεξεργασία του σε RDD εισάγει ανυπέρβλητα προβλήματα λόγω κρυφών χαρακτήρων (`\r`, `\t`, whitespaces) και Null τιμών (`NoneType`), που οδηγούν σε σφάλματα τύπου `TypeError` και `ValueError` στην Python. Η χρήση του DataFrame API για το αρχικό στάδιο επέτρεψε στους native Java/Scala parsers του Spark να καθαρίσουν πλήρως τα δεδομένα και να φιλτράρουν τα Nulls πριν την low-level επεξεργασία. Όσον αφορά τους χρόνοπυς εκτέλεσης παρατηρείται ότι το RDD API ολοκληρώθηκε σε λιγότερο χρόνο (24.24 s) έναντι του DataFrame API (49.58 s). Η συμπεριφορά αυτή εξηγείται από το γεγονός ότι το DataFrame API επωμίστηκε ολόκληρο το κόστος του αρχικού HDFS I/O, του GeoJSON flattening (μέσω της `F.explode`) καθώς και του απαραίτητου `repartition(12)` που έγινε για τη σωστή κατανομή της μνήμης στο cluster.Συμπερασματικά μ[πορούμε να πούμε ότι ο συνδυασμός του DataFrame API για τη στιβαρή διαχείριση, ανάγνωση και φιλτράρισμα δομημένων αρχείων, μαζί με το RDD API για in-memory μετασχηματισμούς, αποτελεί μια εξαιρετική πρακτική. Επιβεβαιώνεται ότι το Spark διαχειρίζεται το I/O και το schema enforcment πολύ πιο αποδοτικά από την Python, ενώ η Python μπορεί να εκτελέσει γρήγορα in-memory low-level operations όταν τα κλειδιά της είναι απόλυτα καθαρισμένα.

## Ζητούμενο 5ο: Υλοποίηση Query 4

**Run A1**

- Αριθμός Executors: 2
- Cores ανά Executor: 1
- Μνήμη ανά Executor: 2 GB
- Συνολικός Χρόνος Εκτέλεσης: 82.59 seconds

Ο παρακάτω πίνακας παρουσιάζει τη μέση απόσταση των εγκλημάτων από το πλησιέστερο αστυνομικό τμήμα ανά (division):

| **Division** | **Average Distance (km)** | **Count (#)** |
|---|---|---|
| HOLLYWOOD | 2.074 | 224,124 |
| VAN NUYS | 2.939 | 208,129 |
| SOUTHWEST | 2.191 | 189,126 |
| WILSHIRE | 2.593 | 186,376 |
| 77TH STREET | 1.717 | 170,620 |
| NORTH HOLLYWOOD | 2.642 | 168,096 |
| OLYMPIC | 1.729 | 162,805 |
| PACIFIC | 3.854 | 162,027 |
| CENTRAL | 0.993 | 154,689 |
| RAMPART | 1.534 | 153,204 |
| SOUTHEAST | 2.444 | 143,803 |
| WEST VALLEY | 3.022 | 136,622 |
| FOOTHILL | 4.260 | 132,482 |
| TOPANGA | 3.297 | 131,054 |
| HARBOR | 3.702 | 127,071 |
| HOLLENBECK | 2.677 | 116,235 |
| WEST LOS ANGELES | 2.790 | 115,969 |
| NEWTON | 1.635 | 111,392 |
| NORTHEAST | 3.623 | 108,243 |
| MISSION | 3.676 | 97,926 |
| DEVONSHIRE | 2.825 | 77,180 |

Table 8: Μέση απόσταση εγκλήματος-σταθμού ανά αστυνομικό τομέα στο LA.

Σύμφωνα με το Physical Plan που παρήγαγε η μέθοδος explain(), ο Catalyst Optimizer του Spark επέλεξε by default τη στρατηγική BroadcastNestedLoopJoin (BuildRight, Cross). Καθώς το query εκτελεί ένα CROSS JOIN ανάμεσα στον πολύ μεγάλο πίνακα των εγκλημάτων (`raw_crimes`) και τον πολύ μικρό πίνακα των αστυνομικών σταθμών (`raw_stations`), το Spark επέλεξε ορθά να κάνει broadcast τον μικρό πίνακα σε όλους τους executors. Με αυτόν τον τρόπο, αποφεύγεται η μετακίνηση των εκατομμυρίων εγγραφών των εγκλημάτων μέσω του δικτύου, κάτι το οποίο θα δημιουργούσε shuffle overhead, και τελικά βελτιώνει σημαντικά την απόδοση της εκτέλεσης.

**Run A2**

- Αριθμός Executors: 2
- Cores ανά Executor: 2
- Μνήμη ανά Executor: 4 GB
- Συνολικός Χρόνος Εκτέλεσης: 44.47 seconds

Από το Run A1 στο Run A2, πραγματοποιήθηκε κάθετη κλιμάκωση. Διπλασιάστηκαν τα CPU cores και η διαθέσιμη μνήμη ανά executor. Με την αλλαγή αυτή ο χρόνος εκτέλεσης μειώθηκε αισθητά από τα 82.59s στα 44.47s (μείωση κατά περίπου 46.1%). Η τεράστια αυτή βελτίωση επιβεβαιώνει την υψηλή αποδοτικότητα του κώδικα PySpark Dataframe/SQL API. Λόγω των 2 cores ανά executor, το Spark ήταν σε θέση να εκτελεί 4 tasks ταυτόχρονα στο cluster, μειώνοντας το συνολικό serialization και scheduling overhead των tasks. Η στρατηγική του Optimizer όπως ήταν λογικό παρέμεινε σταθερό (BroadcastNestedLoopJoin) λόγω του μεγέθους των αρχείων, και επιβεβαιώνει ότι η βελτίωση της ταχύτητας οφείλεται αποκλειστικά στην αλλαγή των configurations.

**Run A3**

- Αριθμός Executors: 2
- Cores ανά Executor: 4
- Μνήμη ανά Executor: 8 GB
- Συνολικός Χρόνος Εκτέλεσης: 30.21 seconds

Παρά τη σημαντική μείωση του χρόνου στο Run A3 (30.21s από τα 44.47s του run A2), παρατηρείται φθίνουσα οριακή απόδοση (diminishing returns). Το cost του driver για τον συντονισμό και το serialization των δεδομένων αρχίζουν να αποτελούν το κυρίαρχο bottleneck, περιορίζοντας τη δυνατότητα περαιτέρω γραμμικής επιτάχυνσης.

Σαν συνολική αποτίμηση για την ομάδα A (Vertical Scalability με σταθερούς 2 executors με αυξανόμενους πόρους) μπορούμε να πούμε πως η σημασία του επαρκούς παραλληλισμού είναι τεράστια. Καθώς πηγαίονουμε από το 1 core (82.59s) στα 2 cores (44.47s) και τελικά στα 4 cores (30.21s), η υπολογιστική ισχύς ανά διεργασία αυξάνεται, επιτρέποντας στο Spark να εκτελεί ταυτόχρονα περισσότερα tasks. Η βελτίωση είναι εμφανής και εντυπωσιακή.

**Run B1**

Πριν ξεκινήσει η μελέτη για το Horizontal Scaling (ομάδα B) πρέπει να αναφερθεί ότι το Run A3 της Ομάδας A και το Run B1 της Ομάδας B χρησιμοποιούν το ίδιο ακριβώς configuration. Και στα δύο runs ισχύει ότι:

$$\text{Total Cores} = 2 \text{ executors} \times 4 \text{ cores/executor} = 8 \text{ cores} \tag{1}$$

$$\text{Total Memory} = 2 \text{ executors} \times 8 \text{ GB/executor} = 16 \text{ GB} \tag{2}$$

Άρα το Run B1 είναι ταυτόσημο με αυτή του Run A3 και επομένως χρησιμοποιείται ο ήδη καταγεγραμμένος χρόνος εκτέλεσης των 30.21 δευτερολέπτων.

**Run B2**

- Αριθμός Executors: 4
- Cores ανά Executor: 2
- Μνήμη ανά Executor: 4 GB
- Συνολικός Χρόνος Εκτέλεσης: 32.61 seconds

Στο Run B2, οι σταθεροί πόροι των 8 cores και 16 GB RAM ανακατανεμήθηκαν οριζόντια (Horizontal Scaling), διπλασιάζοντας τον αριθμό των num-executors και υποδιπλασιάζοντας τους πόρους. Τα αποτελέσματα έδειξαν μικρή αύξηση του χρόνου εκτέλεσης στα 32.61s (αύξηση κατά 7.9% σε σχέση με τα 30.21s του Run B1), κάτι το οποίο όμως είναι αναμενόμενο σε distributed systems. Παρά το γεγονός ότι η συνολική υπολογιστική ισχύς παραμένει ίδια, το coordination και το scheduling overhead του Kubernetes αυξάνονται, όπως αυξάνεται και το network communication κόστος κατά το Shuffle. Η στρατηγική του Optimizer (BroadcastNestedLoopJoin) παρέμεινε πλήρως αποτελεσματική.

**Run B3**

- Αριθμός Executors: 8
- Cores ανά Executor: 1
- Μνήμη ανά Executor: 2 GB
- Συνολικός Χρόνος Εκτέλεσης: 34.05 seconds

Το Run B3 είναι το ακραίο σενάριο του Horizontal Scaling, όπου οι σταθεροί πόροι (8 cores / 16 GB) κατακερματίστηκαν στον μέγιστο δυνατό αριθμό executors (8), με τον καθένα να διαθέτει μόλις 1 core και 2 GB μνήμης. Όπως ήταν λογικό και από το προηγούμενο run, ο χρόνος εκτέλεσης αυξήθηκε περαιτέρω στα 34.05s. Η συμπεριφορά αυτή επιβεβαιώνει την ύπαρξη σημαντικού serialization και coordination overhead. Με 8 ανεξάρτητα pods, το Kubernetes απαιτεί περισσότερο χρόνο για το scheduling και initialization, ενώ το Spark αντιμετωπίζει έντονο overhead κατά το Shuffle για τη συγχώνευση των partial aggregations από 8 διαφορετικά σημεία.

**Γενικό Συμπέρασμα για Scalability**

Η επεξεργασία του Query 4 κάτω από διαφορετικά configurations οδήγησε στα παρακάτω συμπεράσματα:

1. Το Vertical Scaling υπερέχει αισθητά. Η σταδιακή ενδυνάμωση των executors από το 1 core (82.59s) στα 4 cores (30.21s) επέφερε τη δραστικότερη μείωση του χρόνου εκτέλεσης. Αυτό αποδεικνύει ότι το συγκεκριμένο query επωφελείται σημαντικά από την παράλληλη εκτέλεση tasks εντός της ίδιας κοινής μνήμης RAM.
2. Αδυναμία του Horizontal Scaling. Οι πολλοί μικροί executors εισάγουν network overhead κατά το shuffle και υποβαθμίζουν την απόδοση.

Συνεπώς, η βέλτιστη στρατηγική είναι η χρήση λίγων και ισχυρών executors αντί πολλών και μικρών , καθώς μεγιστοποιεί την τοπικότητα της μνήμης (data locality).

## Ζητούμενο 6ο: Ανάλυση του Catalyst Optimizer και Στρατηγικές Join για τα Queries 3,4.

Στο παρόν ζητούμενο εξετάζεται η συμπεριφορά του Catalyst Optimizer του Spark κατά την εκτέλεση του Query 3 (DataFrame API). Χρησιμοποιώντας τη μέθοδο df.explain(mode="formatted"), αποτυπώθηκε το Physical Plan με σκοπό την αναγνώριση της στρατηγικής join που επιλέγεται Default, καθώς και την αξιολόγηση της επίδρασης των Join Hints στην απόδοση του συστήματος.

### Default Run

Κατά την πρώτη εκτέλεση του κώδικα, ο Catalyst Optimizer κατέγραψε χρόνο εκτέλεσης **37.65 δευτερόλεπτα**. Μέσω της `explain()` απομονώθηκε το κρίσιμο κομμάτι του Physical Plan, όπως φαίνεται παρακάτω:

```
+- BroadcastHashJoin Inner BuildRight (15)
   :- Filter (10)
   :  +- HashAggregate (9)
   :  ... [GeoJSON Pipeline]
   +- BroadcastExchange (14)
      +- Project (13)
         +- Filter (12)
            +- Scan csv  (11)
```

Όπως παρατηρείται στο block (15) του Physical Plan, ο Spark επέλεξε αυτόματα τη στρατηγική BroadcastHashJoin Inner BuildRight. Η στρατηγική αυτή επιλέχθηκε πρωτίστως εξαιτίας του μεγέθους των δεδομένων. Ο Catalyst Optimizer αναγνώρισε ότι ο πίνακας που προέκυψε από το καθαρισμένο CSV των εισοδημάτων (`df_income_clean`) είναι εξαιρετικά μικρός (μερικά Kilobytes, BuildRight), σε αντίθεση με τον μεγάλο πίνακα που προέκυψε από το GeoJSON (`df_pop_grouped`).Επομένως, αντί να εκτελέσει full shuffle των δεδομένων και των δύο πινάκων μέσω του δικτύου με βάση το `zip_code`, ο Spark χρησιμοποίησε έναν `BroadcastExchange` (block 14). Με αυτόν τον τρόπο, αντέγραψε ολόκληρο τον μικρό πίνακα του CSV στη μνήμη όλων των executors. Επίσης κάθε executor εκτέλεσε το join τοπικά στη μνήμη του (data locality), συγκρίνοντας τα δικά του partitions του GeoJSON με τον "broadcasted" πίνακα εισοδημάτων, εκμηδενίζοντας το network I/O overhead. Η αυτόματη επιλογή του Ctalyst Optimizer για Broadcast Hash Join αποτελεί την βέλτιστη δυνατή στρατηγική, γεγονός που επιβεβαιώνεται και από την πολύ υψηλή ταχύτητα εκτέλεσης (37.65 δευτερόλεπτα).

### Πείραμα 2: Sort Merge Join

Στο δεύτερο πείραμα, χρησιμοποιήθηκε το Join Hint `.hint("MERGE")` ώστε να υποχρεωθεί ο Catalyst Optimizer να παρακάμψει την επιλογή του Broadcast και να εκτελέσει έναν αλγόριθμο *Sort Merge Join (SMJ)*. Ο χρόνος εκτέλεσης αυξήθηκε αισθητά στα 40.41 δευτερόλεπτα. Το αντίστοιχο Physical Plan από την `explain()` αποτυπώνεται παρακάτω:

```
+- SortMergeJoin Inner (17)
   :- Sort (11)
   :  +- Filter (10)
   :     +- HashAggregate (9)
   :        +- Exchange hashpartitioning(zip_code#12, 200) (8)
   +- Sort (16)
      +- Exchange hashpartitioning(zip_code#47, 200) (15)
         +- Project (14)
```

Όπως εμφανίζεται και παραπάνω, η εισαγωγή του hint άλλαξε ριζικά τη στρατηγική επεξεργασίας. Στα blocks (8) και (15), ο Spark αναγκάστηκε να εκτελέσει full shuffle και για τα δύο DataFrames μέσω του δικτύου

(hashpartitioning σε 200 partitions), προκειμένου να διασφαλίσει ότι οι εγγραφές με τα ίδια zip_code θα καταλήξουν στους ίδιους executors. Επιπλέον στα blocks (11) και (16), πριν την ένωση, το σύστημα κατανάλωσε υπολογιστικούς πόρους (CPU/Memory) για να ταξινομήσει τα δεδομένα και των δύο πλευρών σε αύξουσα σειρά (ASC NULLS FIRST). Όλη αυτή η διαδικασία οδήγησε πτοφανώς και σε αύξηση του χρόνου εκτέλεσης κατά περίπου 3 δευτερόλεπτα κάτι που επιβεβαιώνει και τη θεωρία των κατανεμημένων συστημάτων. Για ασύμμετρα dataset , η στρατηγική Sort Merge Join εισάγει άσκοπο sorting overhead, καθιστώντας την σίγουρα λιγότερο αποδοτική από το Broadcast Join.

**Πείραμα 3: Shuffled Hash Join**

Στο τρίτο πείραμα, ο Catalyst Optimizer αναγκάστηκε να παρακάμψει τις υπόλοιπες επιλογές και να εκτελέσει έναν αλγόριθμο Shuffled Hash Join. Ο χρόνος εκτέλεσης διαμορφώθηκε στα 42.56 δευτερόλεπτα. Το Physical Plan που παρήγαγε η explain() αποτυπώνεται παρακάτω:

```
+- ShuffledHashJoin Inner BuildRight (15)
   :- Filter (10)
   :  +- HashAggregate (9)
   :     +- Exchange hashpartitioning(zip_code#12, 200) (8)
   +- Exchange hashpartitioning(zip_code#47, 200) (14)
      +- Project (13)
```

Όπως φαίνεται στο physical plan, η χρήση του συγκεκριμένου hint οδήγησε αρχικά στη διατήρηση του Shuffle. Στα blocks (8) και (14), ο Spark πραγματοποίησε full shuffle μέσω του δικτύου ώστε οι εγγραφές με κοινά zip_code να συγκεντρωθούν στους ίδιους executors. Επιπλέον έγινε δημιουργία ενός In-Memory Hash Table (BuildRight). Σε αντίθεση με το Sort Merge Join, η στρατηγική αυτή δεν προχώρησε sorting των δεδομένων. Αντίθετα, ο Spark εκμεταλλεύτηκε το μικρό μέγεθος του df_income_clean και κατασκεύασε ένα Hash Table απευθείας στη μνήμη του executor, κάνοντας stream τον μεγάλο πίνακα από πάνω για το ταίριασμα. Ο χρόνος των **42.56 δευτερολέπτων** δείχνει ότι το Shuffled Hash Join σημείωσε τη χαμηλότερη επίδοση σε σχέση με το Sort Merge Join (40.41s). Αν και θεωρητικά η έλλειψη της φάσης Sorting θα έπρεπε να καθιστά το Shuffled Hash Join ταχύτερο, στην πράξη η ανάγκη για full shuffle και στα δύο DataFrames, σε συνδυασμό με το υπολογιστικό overhead για την κατασκευή του In-Memory Hash Table, οδήγησε σε μικρή επιβάρυνση. Επειδή το dataset των εισοδημάτων είναι πολύ μικρό, η ταξινόμηση στο Sort Merge Join έγινε σχεδόν ακαριαία, και επομένως υπερτερεί τελικά ελαφρώς του Shuffled Hash Join.

**Πείραμα 4: Nested Loop Join**

Στο τέταρτο και τελευταίο πείραμα, έγινε η εκτέλεση ενός αλγορίθμου Nested Loop Join. Ο χρόνος εκτέλεσης ανήλθε στα 44.41 δευτερόλεπτα, με το Physical Plan να αποτυπώνεται παρακάτω:

```
+- CartesianProduct Inner (14)
   :- Filter (10)
   :  +- HashAggregate (9)
   :  ... [GeoJSON Pipeline]
   +- Project (13)
      +- Filter (12)
         +- Scan csv  (11)
```

Όπως εμφανίζεται στο φυσικό πλάνο (block 14), η στρατηγική αυτή λειτουργεί ως εξής: Αρχικά Ο Spark αναγκάζεται να πάρει κάθε μεμονωμένη γραμμή από το DataFrame του GeoJSON και να τη συγκρίνει με το σύνολο των γραμμών του CSV των εισοδημάτων (συμπεριφορά Nested Loop $N \times M$). Ωστόσο αυτό οδηγεί σε υψηλό υπολογιστικό Overhead. Η στρατηγική αυτή δεν χρησιμοποιεί κάποια δομή κατακερματισμού (Hash Table) ούτε εκμεταλλεύεται την ταξινόμηση, με αποτέλεσμα να επιβαρύνει σημαντικά τη CPU για την εκτέλεση εκατοντάδων επαναλαμβανόμενων συγκρίσεων.Ο χρόνος των 44.41 δευτερολέπτων επιβεβαιώνει ότι η προσέγγιση αυτή είναι εξαιρετικά ασύμφορη για τυπικά Equi-Joins.

### Συγκριτική Αξιολόγηση Στρατηγικών Join

Παρακάτω συγκεντρώνονται οι χρόνοι εκτέλεσης των 4 διαφορετικών στρατηγικών join που δοκιμάστηκαν:

Table 9: Συγκριτικοί χρόνοι εκτέλεσης στρατηγικών Join στον Catalyst Optimizer

| Στρατηγική Join | Χρόνος Εκτέλεσης (seconds) | Απόδοση |
|---|---|---|
| Broadcast Hash Join (Default) | 37.65 | Βέλτιστη (100%) |
| Sort Merge Join | 40.41 | -7.3% |
| Shuffled Hash Join | 42.56 | -13.0% |
| Cartesian Product | 44.41 | -17.9% |

Με βάση τα πειραματικά δεδομένα, η καταλληλότερη στρατηγική join είναι με διαφορά το Broadcast Hash Join, η οποία επιλέχθηκε και αυτόματα από τον Catalyst Optimizer ως default. Η επιλογή αυτή δικαιολογείται πλήρως από την ασυμμετρία των δύο datasets: το αρχείο των εισοδημάτων είναι πολύ μικρό (μερικά KB), επιτρέποντας στον Spark να το αποστείλει ολόκληρο σε όλους τους executors. Με αυτόν τον τρόπο, αποφεύγεται πλήρως η πιο ακριβή διεργασία που είναι το Shuffle,η μεταφορά δηλαδή των δεδομένων μέσω του δικτύου. Αντίθετα, οι στρατηγικές Sort Merge και Shuffled Hash ανάγκασαν το σύστημα να κάνει άσκοπο shuffle των δεδομένων στο δίκτυο, ενώ η SHUFFLE_REPLICATE_NL προκάλεσε υπολογιστική συμφόρηση λόγω της φύσης του nested loop.

Τώρα έχοντας ολοκληρώσει το ζητούμενο για το Query 3, θα εξεταστεί η συμπεριφορά του Catalyst Optimizer του Spark κατά την εκτέλεση του Query 4

### Default Run (Broadcast Join)

Κατά την πρώτη εκτέλεση του κώδικα με τις default ρυθμίσεις, ο Catalyst Optimizer κατέγραψε χρόνο εκτέλεσης 30.21 δευτερόλεπτα. Μέσω της explain() απομονώθηκε το κρίσιμο κομμάτι του Physical Plan, όπως φαίνεται παρακάτω:

```
+- BroadcastNestedLoopJoin BuildRight, Cross
   :- Union
   :  :- Filter
   :  :  +- FileScan parquet (raw_crimes)
   :  +- Filter
   :     +- FileScan parquet (raw_crimes)
   +- BroadcastExchange IdentityBroadcastMode
      +- Filter
         +- FileScan csv (raw_stations)
```

Όπως αναφέρθηκε και στο Ζητούμενο 5, ο πίνακας raw_stations είναι εξαιρετικά μικρός, το Spark επιλέγει αυτόματα να τον κάνει BroadcastExchange σε όλους τους executors. Με αυτόν τον τρόπο, η σύνδεση των

δεδομένων πραγματοποιείται τοπικά χωρίς να προκληθεί full shuffle των εκατομμυρίων εγγραφών των εγκλημάτων, γεγονός που εξηγεί τον πολύ χαμηλό χρόνο εκτέλεσης (30.21s).

**Πείραμα 2: Nested Loop Join**

Για τη μελέτη της κατανεμημένης στρατηγικής Shuffle Replicate Nested Loop Join, χρησιμοποιήθηκε το SQL hint /*+ SHUFFLE_REPLICATE_NL(c, s) */, διατηρώντας αναλλοίωτη τη γεωγραφική λογική του CROSS JOIN. Ο Catalyst Optimizer κατέγραψε χρόνο εκτέλεσης 32.89 δευτερόλεπτα και παρήγαγε το ακόλουθο Physical Plan:

```
== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=false
+- Sort [##183L DESC NULLS LAST], true, 0
   +- Exchange rangepartitioning(##183L DESC NULLS LAST, 200)
      +- HashAggregate(keys=[station_division#170], functions=[avg(...)])
         +- Exchange hashpartitioning(station_division#170, 200)
            +- HashAggregate(keys=[station_division#170], functions=[partial_avg(...)])
               +- Project [station_division#170, distance_km#171]
                  +- Filter (rnk#175 = 1)
                     +- Window [row_number() ...]
                        +- CartesianProduct
                           :- Union
                           :  :- Filter
                           :  :  +- FileScan parquet [DR_NO#0,LAT#26,LON#27] (raw_crimes)
                           :  +- Filter
                           :     +- FileScan parquet [DR_NO#56,LAT#82,LON#83] (raw_crimes)
                           +- Filter
                              +- FileScan csv [X#157,Y#158,DIVISION#160] (raw_stations)
```

Με την επιβολή του hint, ο Optimizer εξαναγκάζεται να παρακάμψει τη στρατηγική Broadcast και εκτελεί `CartesianProduct`. Στη φάση αυτή, πραγματοποιείται full distributed shuffle και replication των partitions στο δίκτυο της συστάδας, ώστε κάθε executor να αποκτήσει όλα τα απαραίτητα ζεύγη. Η εισαγωγή του overhead για τα 2+ εκατομμύρια εγγραφών εξηγεί την αύξηση του χρόνου εκτέλεσης στα 32.89s (σε σχέση με τα 30.21s του default broadcast), επιβεβαιώνοντας κάτι το οποίο είναι ήδη γνωστό, ότι καθιστώντας σαφή την επικράτηση της στρατηγικής Broadcast όταν ο ένας πίνακας είναι επαρκώς μικρός, η επικρατέστερη στρατηγική είναι αυτή του Broadcast

**Αδυναμία Εκτέλεσης των Στρατηγικών MERGE και SHUFFLE_HASH**

Κατά την προσπάθεια εξαναγκασμού των κατανεμημένων στρατηγικών `Sort-Merge Join` (μέσω του hint `/*+ MERGE */`) και `Shuffle Hash Join` (μέσω του hint `/*+ SHUFFLE_HASH */`), ο Catalyst Optimizer του Spark απέρριψε ρητά την εκτέλεσή τους, επιστρέφοντας το ακόλουθο μήνυμα στα logs:

`WARN HintErrorLogger: Hint (strategy=merge) is not supported in the query: no equi-join keys.`

Οι αλγόριθμοι `Sort-Merge Join` και `Shuffle Hash Join` έχουν σχεδιαστεί αποκλειστικά για την επίλυση Equi-Joins (`c.column = s.column`). Συγκεκριμένα:

- Για το Sort-Merge Join απαιτείται η ύπαρξη κλειδιού ισότητας προκειμένου να εκτελεστεί `hashpartitioning` στο δίκτυο και, στη συνέχεια, εσωτερική ταξινόμηση (`Sort`) των εγγραφών κάθε partition με βάση την τιμή αυτού του κλειδιού. Στο γεωγραφικό Cross Join του Query 4, δεν υφίσταται τέτοιο κλειδί και επομένως αποτυγχάνει.
- Ομοίως το Shuffle Hash Join απαιτεί και αυτό κλειδί σύνδεσης, καθώς βασίζεται στη δημιουργία ενός Hash Table στη μνήμη για τον μικρότερο πίνακα, ώστε να γίνει η αντιστοίχιση με τον μεγάλο πίνακα. Χωρίς κλειδί, η συνάρτηση κατακερματισμού δεν έχει όρισμα για να παράγει διευθύνσεις μνήμης.

Στο Query 4, η ανάγκη για τον εντοπισμό του πραγματικά πλησιέστερου σταθμού απαιτεί τον γεωγραφικό έλεγχο κάθε εγκλήματος με το σύνολο των σταθμών, ανεξάρτητα από τα όρια των divisions. Επομένως αν χρησιμοποιούσαμε σαν equi-key τα divisions (AREA/PREC), θα αλλοιωνόταν ριζικά το αποτέλεσμα και θα περιόριζε τις συγκρίσεις μόνο ανά περιοχή, οδηγώντας σε τελείως λάθος συμπεράσματα αφού ένα έγκλημα θα μπορούσε να διαπραχθεί σε ένα division αλλά να είναι κοντινότερα σε ένα αστυνομικό τμήμα διαφορετικού division. Συνεπώς, για το συγκεκριμένο query, το Spark είναι αρχιτεκτονικά περιορισμένο να υποστηρίζει μόνο Broadcast Nested Loop (Run 1) και Cartesian Product (Run 2). Οποιοδήποτε άλλο hint αγνοείται αυτόματα, αναγκάζοντας το Physical Plan να επιστρέψει σε μία από τις δύο αυτές μεθόδους.